

Compute layers assigned to this rank

Send to the next rank (asynchronously)

(Not handled: overlapping communication/computation to eliminate pipeline bubbles)

(Interleave ~~for~~ forward and backward passes)

backwards pass: --- ~~idle~~

clean-up: ---

- 1: Receiving gradients from the next rank
- 2: computing ~~for~~ the local gradients with activations
- 3: sending gradients to the previous rank

Question: ask chatGPT

How micro-batch splits reduce the rank idle time.

during the secondary ^{pipeline} parallelization forward/backward passes?

without micro-batching: $\Downarrow$

Time step	GPU0 (Rank=0)	GPU1 (Rank=1)
$t=1$	compute layers 1-2	Idle state
$t=2$	idle status	compute layers 3-4

$\Rightarrow$ only one GPU works at a time - total utilization $\approx 50\%$

this idle time is the pipeline bubble - the other devices wait for data

with micro-batching: $\Downarrow$

Now, we split the batch into multiple micro-batches

say we use 4 micro-batches: M1, M2, M3, M4.

Time step	GPU 0 (Rank #0)	GPU #1 (Rank #1)
t=1	compute M1 - layers 1-2	Idle status
2	M2 - layers 1-2	M1 layers 3-4
3	M3 - layers 1-2	M2 layers 3-4
4	M4 - layers 1-2	M3 layers 3-4
5	Idle	M4 layers 3-4

Now: Both GPUs are working in parallel for most of time

$\Rightarrow$ GPU 0 works on the next micro-batch

while $\Rightarrow$ GPU 1 process the previous micro-batch's activation

$\Rightarrow$ This overlapping reduces the idle gap between stages,

only a few first/last steps have idle GPUs

How to calculate Bubble fraction (fraction of idle time):

$$\text{Bubble Fraction} = \frac{p-1}{\text{num of micro-batch per batch} \leftarrow m + p - 1} \times \text{num of pipeline stage}$$

$\downarrow$
 num of pipeline stage

interleaves
GPU #1/#2 $\Rightarrow$

p	M	Bubble fraction	Utilization
2	1	50%	50%
2	4	20%	80%
4	16	15%	85%

Trade-off: { more micro-batch: better overlapping [comm/comp], high utilization
 too many micro-batch: more comm/memory overhead, low efficient kernel